Web Server Attacks

Concepts, Attack Methodology, Tools & Countermeasures

CEH Module 13 - Study Notes

Contents

1. Web Server Concepts
2. Web Server Attacks
3. Web Server Attack Methodology
4. Web Server Attack Tools
5. Countermeasures
6. Patch Management
7. Web Server Security Tools
8. Lab: Information Gathering using Ghost Eye
9. Lab: Webserver Reconnaissance using Skipfish
10. Lab: Footprinting with httprecon, ID Serve, Netcat
11. Lab: Cracking FTP Credentials using a Dictionary Attack

Disclaimer: These notes are for authorized educational and certification-study purposes only (e.g., CEH exam prep / lab environments such as vLabs). Only perform these techniques against systems you own or are explicitly authorized to test.

1. Web Server Concepts

What is a Web Server?

A web server is a program (running on top of an OS) that uses HTTP/HTTPS to serve files and application content to clients (browsers) on request. Examples: Apache HTTP Server, Microsoft IIS, Nginx, Apache Tomcat, LiteSpeed.

Web Server Architecture (Components)

- Hardware / OS layer - the underlying server and operating system
- Web server software - Apache, IIS, Nginx, Tomcat, etc.
- Modules / plugins / extensions - third-party add-ons (PHP, mod_ssl, etc.)
- Hosted web application - the actual site/application logic
- Database - backend data store the application talks to

Why Web Servers Get Compromised

- Improper/default server configuration (sample files, default accounts)
- Unpatched OS, server software, or installed modules
- Lack of a proper security policy, procedures, and maintenance plan
- Misconfigured SSL certificates and encryption settings
- Unnecessary default/sample content, services, and open ports left enabled
- Lack of security awareness among administrators (poor password practices, no monitoring)

Impact of Web Server Attacks

- Compromise of user/customer accounts and data
- Website defacement, reputational damage
- Data theft, secondary attacks against the organization's internal network
- Root/administrative access to the server, financial losses

2. Web Server Attacks

DoS / DDoS Attacks

Flooding the server with traffic/requests (e.g., SYN flood, ping of death, application-layer floods) so it cannot serve legitimate users.

DNS Server Hijacking

Compromising the DNS server to redirect requests for the legitimate domain to a malicious/rogue server.

DNS Amplification Attack

Abusing open DNS resolvers with spoofed source IP to flood a victim with large DNS responses.

Directory Traversal Attacks

Using '../' sequences (or encoded variants) to access files/directories outside the intended web root.

Man-in-the-Middle / Sniffing Attacks

Intercepting traffic between client and server (especially over unencrypted HTTP) to steal credentials/session tokens.

Phishing Attacks

Luring users to a fake version of the site (often hosted after server compromise) to steal credentials.

Website Defacement

Altering the visual appearance/content of a website after gaining unauthorized access.

Web Server Misconfiguration

Exploiting default accounts/passwords, directory listing, verbose error pages, unnecessary services.

HTTP Response Splitting

Injecting data into HTTP headers to split a single response into multiple attacker-controlled responses.

Web Cache Poisoning

Replacing cached content with malicious content so it is served to subsequent visitors.

SSH Brute Force Attack

Repeatedly guessing SSH credentials to gain remote shell access.

Web Application Attacks (via server)

SQL Injection, XSS, session hijacking, etc. launched against applications hosted on the server.

3. Web Server Attack Methodology

Attackers (and pentesters, with authorization) typically follow a structured sequence of phases:

Step 1 - Information Gathering

Whois, DNS lookups, OSINT, search-engine queries, SSL certificate info, archive.org history, locating subdomains and related infrastructure. Tools: Ghost Eye, Whois, Netcraft.

Step 2 - Web Server Footprinting

Identifying server banner, type, version, OS, and installed modules via banner grabbing and tool-based fingerprinting. Tools: httprecon, ID Serve, Netcat, Telnet.

Step 3 - Mirroring a Website

Downloading a full copy of a website locally (e.g., with HTTrack) to browse offline and analyze structure/code/comments without repeatedly hitting the live server.

Step 4 - Vulnerability Scanning

Scanning the server/application for known vulnerabilities and misconfigurations. Tools: Nikto, Skipfish, Nessus, OpenVAS.

Step 5 - Session Hijacking

Capturing or predicting valid session tokens/cookies to impersonate an authenticated user.

Step 6 - Password/Credential Attacks

Brute-force or dictionary attacks against exposed services (FTP, SSH, RDP, admin panels) to obtain valid credentials.

4. Web Server Attack Tools

Category	Example Tools	Purpose
Recon / OSINT	Ghost Eye, Whois, Netcraft, Shodan	Gather public info, DNS, hosting, tech stack
Footprinting	httprecon, ID Serve, Netcat, Telnet	Identify server type/version via banners
Vulnerability Scanning	Nikto, Skipfish, Nessus, OpenVAS	Detect known vulns & misconfigurations
Exploitation Frameworks	Metasploit	Exploit known vulnerabilities (authorized use)
Web App Testing	Burp Suite, OWASP ZAP	Intercept/test web app traffic and inputs
Password/Brute Force	Hydra, Medusa, Ncrack	Dictionary/brute-force attacks on login services
Mirroring	HTTrack	Clone a website for offline analysis

Note: Many of these are dual-use tools. Legitimate security teams use them for authorized penetration testing and vulnerability assessment under a signed scope/agreement.

5. Countermeasures

Placement & Network Design

- Place web servers in a DMZ, segmented from the internal network
- Use a reverse proxy / load balancer in front of the real server
- Deploy a Web Application Firewall (WAF) such as ModSecurity
- Restrict access with firewalls/ACLs to only required ports/services

Server Hardening

- Remove/disable default accounts, sample files, and unused modules
- Disable directory listing and verbose/debug error messages
- Run the web server process with least-privilege (non-root/non-admin) account
- Disable unused HTTP methods (e.g., TRACE, PUT, DELETE) where not needed
- Enforce strong TLS configuration (disable weak ciphers/old TLS versions)

Monitoring & Response

- Enable centralized logging and forward to a SIEM
- Set up file integrity monitoring (detect defacement/tampering)
- Configure intrusion detection/prevention (IDS/IPS)
- Regularly review access and error logs for anomalies

Authentication & Access

- Enforce strong password policies and MFA for admin access
- Disable or tightly restrict FTP; prefer SFTP/FTPS
- Rotate credentials, remove unused accounts
- Apply account lockout after repeated failed login attempts

6. Patch Management

Patch management is the structured process of identifying, acquiring, testing, and deploying patches/updates to close known vulnerabilities before attackers can exploit them.

- Maintain an up-to-date inventory of all servers, OS versions, and installed software/modules
- Subscribe to vendor security advisories and CVE feeds
- Test patches in a staging environment before production rollout
- Define a patch deployment schedule (critical patches expedited, routine patches on a cycle)
- Use centralized patch management tools (e.g., WSUS, SCCM, or equivalent for Linux - yum/apt with change control)
- Maintain rollback plans in case a patch causes instability
- Document and audit patch compliance regularly

7. Web Server Security Tools

Nikto

Open-source web server scanner that checks for dangerous files, outdated versions, and misconfigurations.

Skipfish

Active reconnaissance/vulnerability-mapping tool from Google; fast crawl that builds an interactive sitemap annotated with security observations.

Nessus / OpenVAS

Full vulnerability scanners covering OS, services, and web server-level vulnerabilities.

ModSecurity

Open-source Web Application Firewall (WAF) module for Apache/Nginx/IIS.

Qualys / Acunetix

Commercial vulnerability management and web app scanning platforms.

8. Lab: Information Gathering using Ghost Eye

Objective

Use Ghost Eye (a Python-based OSINT/information-gathering tool) to collect public information about a target domain/organization - WHOIS data, DNS records, subdomains, geo-IP info, and more.

General Workflow

- Clone/install Ghost Eye on an attacker machine (e.g., Kali/Parrot) - requires Python 3
- Launch the tool from a terminal
- Select the target domain or IP address
- Choose the information-gathering module: WHOIS lookup, DNS lookup, subdomain enumeration, Geo-IP, etc.
- Review the generated report for exposed infrastructure details

Example command pattern:

```
git clone https://github.com/BatMrE/Ghost-Eye.git
cd Ghost-Eye
pip3 install -r requirements.txt
python3 ghost-eye.py
```

What to look for: registrar & registrant info, name servers, mail servers (MX), associated subdomains, and any technology fingerprints that hint at the web server/CMS in use.

9. Lab: Webserver Reconnaissance using Skipfish

Objective

Use Skipfish to actively crawl a target web server and generate an interactive sitemap highlighting potential security issues (e.g., exposed admin paths, outdated software signatures, weak configurations).

General Workflow

- Ensure Skipfish is installed (commonly pre-installed on Kali Linux)
- Create an output directory for the scan report
- Run Skipfish against the target URL with output directed to that folder
- Let the crawl complete (can take time depending on site size)
- Open the generated index.html report in a browser to review findings

Example command:

```
skipfish -o /home/user/skipfish_report http://<target-ip-or-domain>
```

The HTML report groups results by severity/ratio and lets you drill into each discovered path, response code, and flagged issue.

10. Lab: Footprint a Webserver (httprecon, ID Serve, Netcat)

A) httprecon

A Windows GUI tool that fingerprints a web server by analyzing subtle differences in HTTP responses (header order, banner strings, error-page behavior) to determine server software and version with higher accuracy than a simple banner grab.

- Launch httprecon.exe
- Enter the target hostname/IP and port (e.g., 80 or 443)
- Click 'Analyze' to send a series of probe requests
- Review the fingerprint report and confidence-ranked server-type match

B) ID Serve

A lightweight Windows utility (by GRC) that identifies the make/version of a web server (or any internet server) by querying it directly and reading the returned header/banner information.

- Open ID Serve
- Enter the target website URL or IP in the query field
- Click 'Query The Server'
- Read the 'Server Identification' tab for the returned banner/version string

C) Netcat (manual banner grabbing)

Netcat can be used to manually open a raw TCP connection to the web server and send a crafted HTTP request to view the raw response headers (including the Server: banner).

```
nc -v <target-ip> 80
HEAD / HTTP/1.0
(press Enter twice)
```

The response will include headers such as 'Server: Apache/2.4.x' or 'Server: Microsoft-IIS/10.0', confirming the server software/version.

Tip: Banners can be spoofed/hidden as a countermeasure (e.g., ServerTokens/ServerSignature directives in Apache), so cross-check findings across multiple tools/techniques.

11. Lab: Crack FTP Credentials using a Dictionary Attack

Objective

Demonstrate how weak FTP credentials can be recovered using a dictionary (wordlist-based) attack - commonly performed in labs with Hydra against a test FTP server, to illustrate why default/weak credentials and unencrypted FTP are a risk.

General Workflow (example using Hydra)

- Identify the target FTP server IP and confirm port 21 is open (e.g., via Nmap)
- Prepare or choose a username list and a password wordlist (e.g., rockyou.txt)
- Run Hydra pointing it at the FTP service with the chosen wordlists
- Hydra iterates through credential combinations until a valid login is found
- Verify the discovered credentials by manually logging in via FTP

Example command:

```
hydra -L usernames.txt -P passwords.txt ftp://<target-ip>
```

Or for a single known username:

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt ftp://<target-ip>
```

Countermeasures for FTP Credential Attacks

- Disable anonymous FTP and default accounts
- Enforce strong, unique passwords and account lockout after failed attempts
- Replace plain FTP with SFTP/FTPS (encrypts credentials in transit)
- Restrict FTP access by IP allow-list / VPN where possible
- Monitor and alert on repeated failed authentication attempts

---

Important: All hands-on labs above (Ghost Eye, Skipfish, httprecon, ID Serve, Netcat, Hydra/FTP dictionary attacks) must only be performed in an authorized lab environment (e.g., CEH practice range, your own test VMs) or against systems you have explicit written permission to test.